APLICACIONES WEB II – TP1

Proyecto: iHouse – Tienda Online de Productos Apple

Integrantes: Mateo Videla y Lino Tulian

1.1 Equipo y roles

Mateo: desarrollo e integración del Back-End, configuración del servidor y conexión con la API. Lino: documentación, pruebas, seguimiento del proyecto y apoyo en la integración. Los roles pueden rotar durante el desarrollo según las tareas.

1.2 Desarrollo del proyecto y justificación

El proyecto iHouse consiste en el desarrollo del Back-End para una tienda online de productos Apple. El Front-End del sitio fue desarrollado previamente en Aplicaciones Web I, por lo que en esta etapa se desarrolla el servidor y se conecta el sitio existente con una fuente de datos externa.

El Back-End será desarrollado utilizando Node.js y Express.js. Los productos serán obtenidos desde MockAPI y el servidor expondrá una ruta propia para que el Front-End pueda solicitar la información. De esta manera, el Back-End funciona como intermediario entre la fuente de datos y el sitio web.

Funcionalidades que se desarrollarán:

- Configuración de un servidor utilizando Node.js y Express.js.
- Publicación del Front-End existente mediante el servidor.
- Conexión del Back-End con MockAPI para obtener los productos.
- Creación de endpoints para acceder a los datos de los productos.
- Integración del Front-End existente con el endpoint del Back-End.
- Manejo básico de errores en las solicitudes.
- Organización modular y separación de responsabilidades.
- Pruebas del servidor, endpoints y catálogo.
- Documentación y seguimiento mediante GitHub y GitHub Projects.

Flujo principal de datos: MockAPI → Back-End Express → /api/productos → Front-End → catálogo.

1.3 Cronograma de avances y desarrollo

Etapa	Actividad	Estado / avance
Análisis y planificación	Análisis de consignas, objetivo del Back-End y organización de tareas.	Realizado
Configuración del entorno	Configuración de Node.js, npm y creación del proyecto Back-End.	Realizado
Servidor Express	Creación y configuración del servidor con Express.js.	Realizado
Integración con MockAPI	Conexión del servidor con la API externa para obtener productos.	Realizado
Creación de endpoints	Implementación de /api/productos para entregar productos al Front-End.	Realizado / en pruebas
Integración Front-End / Back-End	El Front-End solicita productos al Back-End en lugar de acceder directamente a MockAPI.	En desarrollo
Pruebas	Verificación del servidor, endpoints y carga del catálogo.	En desarrollo
Documentación	Actualización del informe, endpoints y presentación.	Pendiente

1.4 Herramienta de seguimiento y control de versiones

Se utiliza GitHub para registrar cambios mediante commits y GitHub Projects para organizar tareas y controlar el avance mediante estados como Backlog, Ready, In Progress, In Review y Done.

1.5 Tecnologías y herramientas

- Node.js: ejecución de JavaScript en el servidor.
- Express.js: creación y administración del servidor y sus rutas.
- MockAPI: fuente externa de datos de productos.
- JavaScript: desarrollo del Back-End e integración con el Front-End.
- GitHub: repositorio y control de versiones.
- GitHub Projects: organización y seguimiento de tareas.

1.6 Endpoints principales

Método	Ruta	Descripción
GET	/api/productos	Obtiene los productos desde MockAPI y los devuelve al Front-End en formato JSON.
GET	/catalogo	Permite acceder al catálogo del sitio mediante el servidor.

El objetivo de esta etapa es que el Front-End existente consuma los datos a través del Back-End, manteniendo separadas las responsabilidades entre interfaz, servidor y fuente de datos.

2.1 Implementación del servidor Node.js y Express

En esta etapa se desarrolló un servidor utilizando Node.js y Express.

Funcionalidades implementadas

- Servidor Express funcionando en localhost:3000.
- Publicación del Front-End mediante Express.
- Organización modular del proyecto.
- Endpoint para obtener productos.

Tecnologías utilizadas

Tecnología	Uso
Node.js	Entorno de ejecución JavaScript.
Express.js	Servidor web.
JavaScript ES Modu	Importación y exportación de módulos.
Fetch API	Consumo de datos desde API.
MockAPI	API REST externa para productos.

2.2 Integración con una API externa (MockAPI)

Para cumplir con la consigna del TP2 se reemplazó el archivo local productos.json por una API REST externa creada en MockAPI.

Funcionamiento

El catálogo realiza una solicitud HTTP utilizando fetch() y obtiene los productos almacenados en MockAPI.

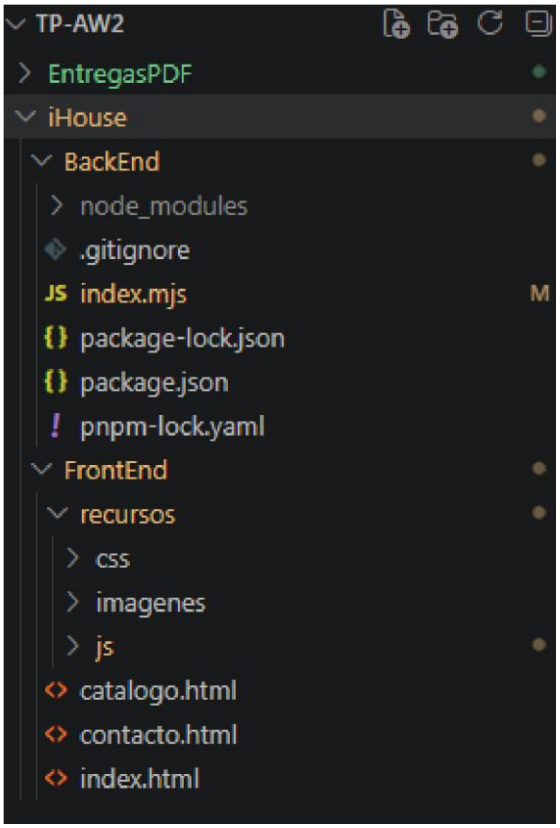
Beneficios

- Los productos se actualizan sin modificar el código.
- Permite administrar el catálogo desde una plataforma externa.
- Facilita futuras operaciones CRUD.

2.3 Organización del proyecto

La aplicación fue organizada separando las responsabilidades entre Front-End y BackEnd.

Estructura del proyecto



Modularización

Archivo	Responsabilidad
index.mjs	Configuración del servidor Express y rutas.
productos.mjs	Datos locales de prueba y endpoint interno.
app.js	Consumo de la API mediante fetch().
renderProductos.js	Renderizado dinámico del catálogo.

2.5.1.3. Endpoints de la API

[GET] /api/productos

Obtiene el listado completo de productos disponibles en el catálogo de iHouse en formato JSON.
Método: GET

Devuelve un arreglo de productos con nombre, descripción, precio, imagen, descuento y oferta.

[GET] /productos (MockAPI)

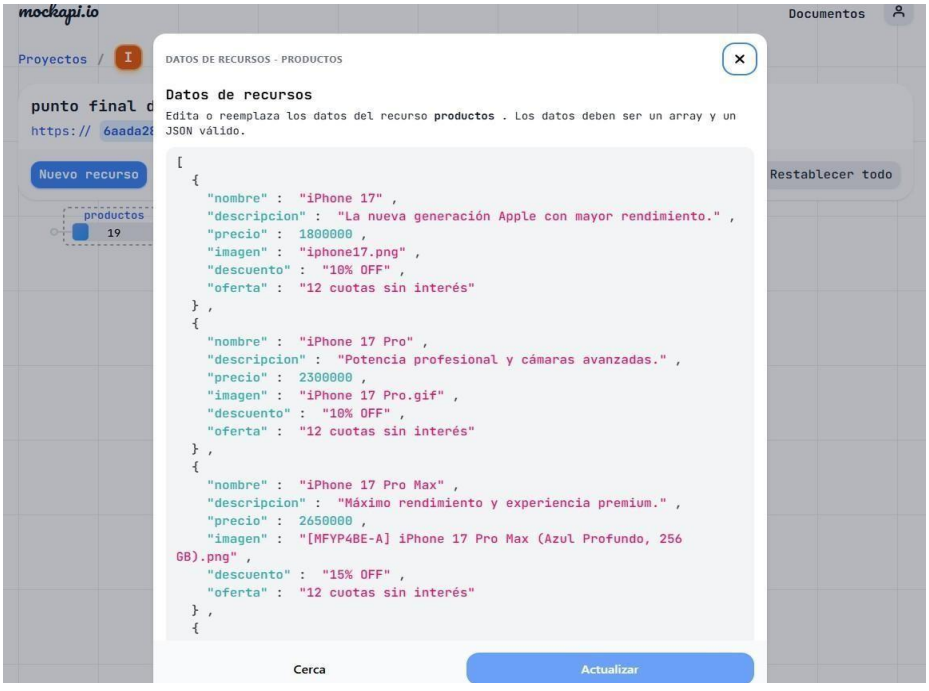
Obtiene el listado de productos almacenados en la API externa MockAPI.

Método: GET

URL base: https://6aada288a2413bf0ec11b843.mockapi.io

Endpoint completo: https://6aada288a2413bf0ec11b843.mockapi.io/productos

Devuelve todos los productos registrados en MockAPI para que el catálogo de iHouse los renderice dinámicamente.



TP3 – Desarrollo del Back-End, API REST y base de datos

3.1. Organización del proyecto mediante MVC

Back-End de iHouse se desarrolla con Node.js y Express, buscando separar las responsabilidades mediante el patrón Modelo-Vista-Controlador (MVC).

El modelo se encarga de interactuar con PostgreSQL, el controlador procesa las solicitudes y devuelve las respuestas, y las rutas definen los endpoints disponibles. La organización por módulos y el uso de Express Router permiten mantener el código ordenado y facilitar su mantenimiento.

3.2. Creación de la API REST

Se desarrolló una API para administrar los productos de iHouse mediante cinco operaciones CRUD:

Método	**Función**
GET	Obtener todos los productos.
GET	Obtener un producto por ID.
POST	Crear un producto.
PUT/PATCH	Modificar un producto.
DELETE	Eliminar un producto.

El endpoint GET /api/productos permite obtener los productos en formato JSON para mostrarlos en el catálogo. Los endpoints de lectura pueden ser consumidos por el Front-End. La configuración de CORS debe verificarse según cómo se ejecute la aplicación.

3.3. Interfaz Front-End

Se integró el catálogo con la API propia mediante JavaScript y fetch(). El archivo renderProductos.js genera dinámicamente las tarjetas con el nombre, precio e imagen de cada producto.

También se ajustaron los estilos CSS para aprovechar mejor el ancho de pantalla y mantener una distribución responsive.

La interfaz de administración para crear, modificar y eliminar productos debe verificarse por separado, ya que el catálogo público no implica necesariamente que esas funciones estén disponibles desde una pantalla administrativa.

3.4. Base de datos PostgreSQL

Se configuró PostgreSQL mediante Docker y se creó la base de datos ihouse, con una tabla productos que contiene los campos id, producto, precio e imagen.

Se cargaron productos de las categorías iPhone, iPad y MacBook. Además, se corrigió el registro de la MacBook de 16 pulgadas, asignándole el nombre correspondiente y la imagen mc16.jpg.

Para cumplir con la entrega, el repositorio debe incluir un archivo SQL que permita recrear la estructura de la base de datos y, cuando corresponda, cargar los datos iniciales.

3.5. Documentación con Swagger

Se incorporó Swagger para documentar y probar los endpoints de la API desde una interfaz interactiva, disponible en /api-docs mientras el servidor está iniciado.

Esta herramienta facilita la comprobación de las operaciones CRUD y permite revisar las solicitudes y respuestas del servidor. Antes de la entrega se debe verificar que los cinco endpoints estén documentados y funcionen correctamente.

3.6. Organización del proyecto

El proyecto se divide en dos partes principales:

- Front-End: HTML, CSS, JavaScript y recursos gráficos.
- Back-End: Node.js, Express, rutas de la API, controladores, modelos y conexión con PostgreSQL.

Docker permite ejecutar la base de datos y Swagger facilita su integración y prueba desde el servidor.

La estructura definitiva debe reflejar la organización por módulos y el uso de Express Router solicitado en la consigna.